

查询处理 Query Processing: Join Algorithms

如何实现基本的关系操作：考虑索引、聚簇等因素，连接的各种实现方案即计算量

提高查询过程性能的思路：操作符的巧妙实现技术、利用关系代数的“等价”、

使用统计数据和本模型进行选择

本节统一使用的案例：

Sailors (sid: integer, sname: string, rating: integer, age: real) Reserves (sid: integer, bid: integer, day: dates, rname: string)

Sailors: Each tuple is 50 bytes long, 80 tuples per page, 500 pages. [S] 表示页面数 pS 表示每个页面的元组数

Reserves: Each tuple is 40 bytes, 100 tuples per page, 1000 pages. [R] 表示页面数 pR 表示每个页面的元组数

cost: 不考虑结果写回的IO，因为去重结果未知

首先考虑如何实现基本的关系操作：

选行 Selection

- 选择度 Selectivity：满足这个条件的元组占有所有元组的比例
- 可选方案：
 - 如果没有索引，遍历一遍：cost = [R]
 - 带有索引（B+树）的选行：两个步骤 $cost = cost_{step_1} + cost_{step_2}$
 1. 使用索引找到满足需要的数据项 Data Entries
 2. 根据数据项找对应的数据记录 Data Record
 - 对于第二步，需要考虑是否聚簇：如果选择度为 10%，对于R表有100页，也就是10000元组命中，对于聚簇的，IO次数为100左右；而对于非聚簇的，如果不使用优化，IO次数为10000左右
 - 带有非聚簇索引（B+树）的选行优化，第二步的实现为：
 1. 对数据项的 RID（页码）进行排序
 2. 按照排序结果来取数据记录

这样，每个数据页只查看一次，尽管此类页面的数量可能比使用聚簇时要高。
- 通常的选行条件：多个条件用 AND / OR 连接
 - 做法：首先转化为合取范式（CNF：析取项和合取）
 - 方法 1：考虑能不能一起查找

1. 找到最便宜的访问路径：IO访问最少（考虑适用哈希还是B+树）
 - 对于B+树：B-tree索引匹配只涉及搜索键前缀中的属性
 - 建立数据项按照A, B, C来排序写作<a, b, c>，如果查询a=5 b=3比较好找，因为是前缀，但直接查b=3不好查询，因为b不是前缀，b的排列被a隔开了。
 - 对于哈希：要有建立在待搜索项上的 Index
 2. 然后根据此找需要的元组
 3. 最后应用任何与索引不匹配的剩余操作
- 方法 2：各自查找再选交集
1. 从每个索引中，获得一组RID
 2. 计算RID集合的交集
 3. 查找这些RID的记录
 4. 最后应用任何未执行的剩余操作

选列 Projection

- 需要考虑重复 Removing Duplicates 的问题：
 - 如果需要去重，则对需要的属性进行排序，排序后线性遍历重复的仅取一个。
 - $cost = cost_{排序} + cost_{遍历}$
 - 在预约表的例子中，假设选择度为0.25，缓冲区为 20 Pages
 - $cost = 1000 + 250 + 2 * 2 * 250 + 250 = 2500$ 次IO
 - 1000 + 250 指取出所有的1000个页面，然后选列（空间只占原来的0.25）故只用250个页面写回
 - \$22250\$ 表示250个页面用20个页面的缓冲区进行外排序，需要两趟，每一趟对所有页面进行一次读一次写（共2次IO）
 - 最后 250 表示遍历一遍做去重需要读入页面数目
- 选列的优化：避免临时文件的硬盘写入 On the fly
 - 排序趟 0：消除不需要的属性数据，只对需要的属性进行排序
 - 排序其余趟：在排序的过程中同时去重
 - 在上面的例子中， $cost = 1000 + 250 + 250 = 1500$ 次IO
 - 趟0：1000次硬盘写入内存，250次为将经筛选的13个排序段写回硬盘
 - 趟1：取出13个排序段（250个页面）并去重（一趟完成）
- 其他选列的技巧
 - 如果一个索引搜索键 Key 包含所有需要的属性：不会有重复，只需一遍索引扫描
 - 如果一个B+树索引搜索关键字前缀有所有想要的属性：按顺序扫描索引，只需按顺序检索数据条目，丢弃不需要的字段，动态地比较相邻元组以检查是否有重复项。

自然连接 Join

- 嵌套循环连接 Nest-Loops Join

- SNLP 简单的嵌套循环连接 $\text{cost}(R \bowtie S) = [R] * [S] * P_R + [R]$ 加号后面是读入 R 的 IO 次数，加号前面是 S 读入 S 的次数。（假定左侧的表为外层循环）

```
foreach tuple r in R do
  foreach tuple s in S do
    if ri == sj then add <r, s> to result
```

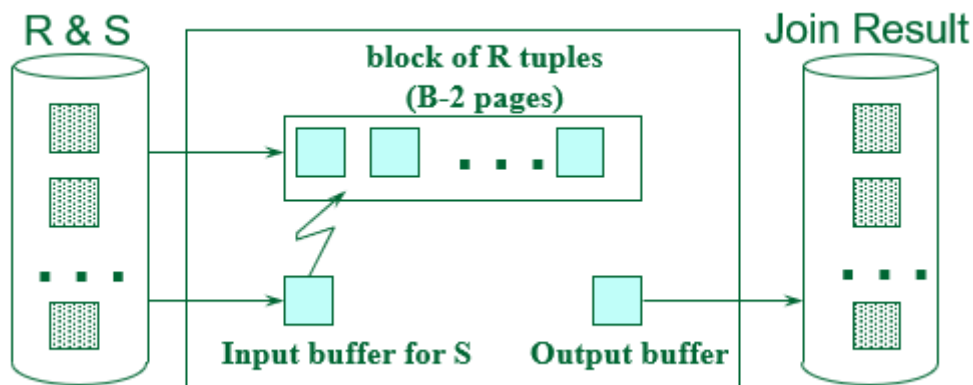
- 基于页面的嵌套循环 Page-Oriented Nested Loops Join

四层循环，先便利页面，再遍历元组

```
foreach page bR in R do
  foreach page bS in S do
    foreach tuple r in bR do
      foreach tuple s in bS do
        if ri == sj then add <r, s> to result
```

$$\text{cost}(R \bowtie S) = [R] * [S] + [R]$$

- 基于块的嵌套循环连接 Block Nested Loops Join



设Buffer有B个页面，选B-2个页面作为R的缓冲取，一个页面作为S的缓冲区，1个为Output，它能够充分利用缓冲区的数目。

- $\text{cost}(R \bowtie S) = [R] + \lceil [R]/(B-2) \rceil * [S]$ (R是外层，S是内层)
- 索引嵌套循环连接 Index-Nested Loops Join (建B+树)
 - 对一个表建树，另一个根据内容查找树，需要考虑是否聚簇。

```
foreach tuple r in R do
  foreach tuple s in S where ri == sj do
    add <r, s> to result
```

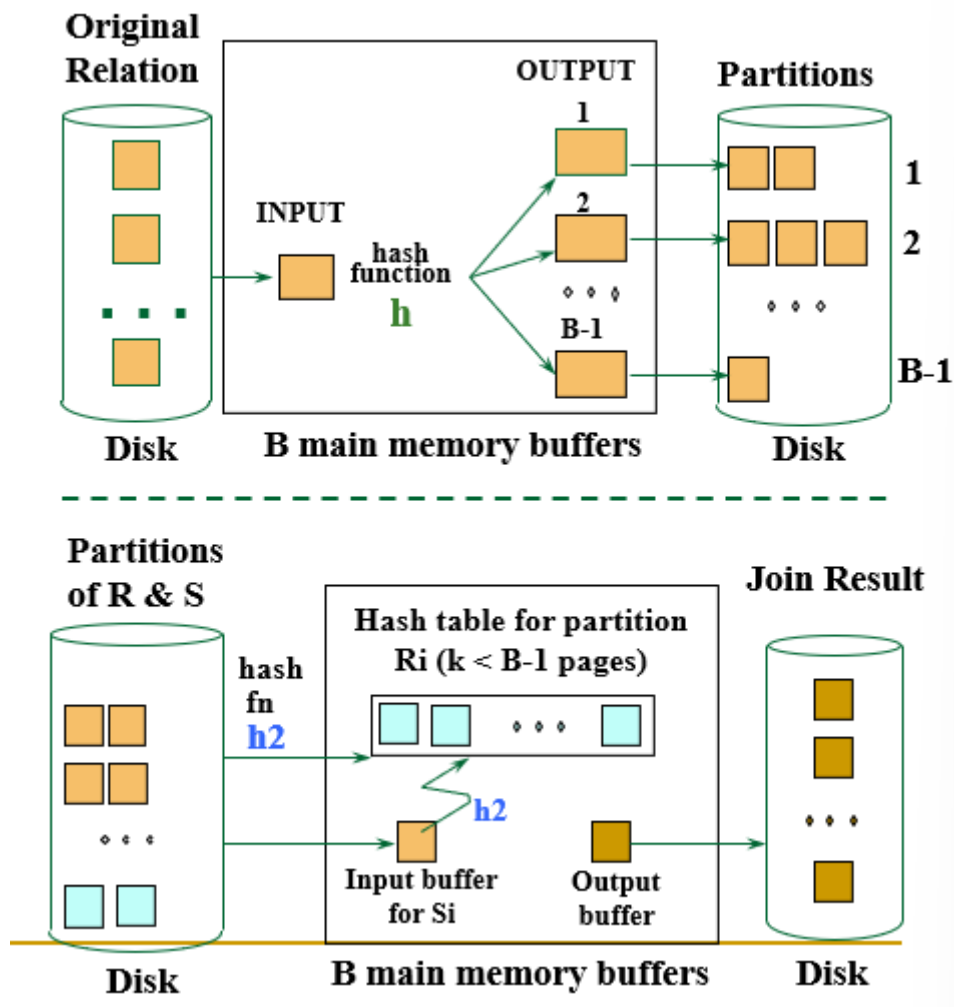
- $\text{cost}(R \Join S) = [R] + [R] * P_R * \text{cost}_{\{\text{find_s_tuples}\}}$
 - $\text{cost}_{\{\text{find_s_tuples}\}}$ 是从根节点遍历树和根据RID获取记录元组的开销
 - 查询RID(s)的成本通常B+树需要2-4个IO。
 - 从RID(s)检索记录的成本取决于是否聚簇：
 - 聚簇索引：匹配的S元组每页1个I/O。
 - 非聚簇索引：每个匹配的S元组最多有1个I/O。

• 排序归并连接 Sort-merge Join

- 先分别排序，然后二路归并（串联 Tandem 两个有序段，寻找匹配的）
- $\text{cost}(R \bowtie S) = \text{cost}(\text{sort}(R)) + \text{cost}(\text{sort}(S)) + [R] + [S]$
 - 即分别排序的读写IO(排序趟数 * 2 * 页面数) + [R] + [S]
 - 会有一种情况：就是两张表已经有序，代价就变成了 [R] + [S]
- 归并排序有一个好处就是让结果是有序的。
- 排序归并连接的优化：**我们可以将R和S排序中的合并阶段与合并所需的连接相结合。**
 - 如果 $B > \sqrt{L}$ ，其中L是较大关系的大小，使用排序在Pass 0中产生长度为两个B的若干有序段，每个关系有序段的数目都小于 $B/2$ 。
 - 在合并阶段：为每一个关系的每个有序段分配1个页面（缓冲区中有一半放R的，一半放S的有序段），然后连接条件时进行合并
 - Cost：第0趟两个表的读写 + 第1趟中两个表的读（我觉得不会有第二趟），所以又等于 $\text{cost}(R \bowtie S) = 3 * ([R] + [S])$

• 哈希连接 Hash-Join

- 只能处理等值连接
- 分桶：使用同样的哈希函数对两个表进行分组 Partitions
 - B个内存页面里拿一个作为输入，B-1个作为数据输出缓冲区，哈希函数为 $\text{hash}(\text{key}) \bmod (B - 1)$ ，每当一个输出缓冲区页面满了，就写到硬盘上
 - 把一个分组（假设分组大小小于 B-2个页面）扔进来，内存临时建立哈希表使用**另一个哈希函数**整到 (B-2) 个输入缓冲区内，剩下一个输入缓冲区和一个输出缓冲区。
 - S的每一个元组经过哈希函数落到B-2个元组中，去探测然后满足条件的扔到输出缓冲区。



- 注：
 - 注意到这里使用了两个哈希函数 h 和 h_2 ，前者用于给两个表分组，后者用于前面分好的组在进行另一种划分和映射
 - 缓冲区的页面有大小限制， $\lceil R \rceil / (B - 1) \leq (B - 2)$ 因此 B 必须要大于 $\sqrt{\lceil R \rceil}$ (倾向于把比较小的表作为用来等待探测的表（就是此处的 R ）)
 - 若每个桶的页面的是不均匀，可以递归的使用哈希。
- 代价估算：哈希阶段分桶： $2(\lceil R \rceil + \lceil S \rceil)$ 次IO，在匹配阶段读入： $\lceil R \rceil + \lceil S \rceil$ ，总次数为 $\text{cost}(R \bowtie S) = 3 * (\lceil R \rceil + \lceil S \rceil)$

• 对比排序归并连接和哈希连接：

- 如果关系大小相差很大，则使用Hash连接。
- Hash Join被证明是高度并行的。
- 排序合并对数据倾斜（Data Skew）不太敏感，结果有序。

其他讨论

- 交集（Intersection）和叉积（Cross-Product）作为连接的特殊情况。
- 联合（Union(Distinct)）和除（Except）相似
- 联合 Union：

- 基于排序的联合方法：对这两个关系进行排序（根据所有属性的组合），扫描排序的关系并合并它们（合并甚至可以从Pass 0就开始）。
- 基于哈希的联合方法：用哈希函数h划分R和S得到多个Partition。对于每个S-Partition，在内存中构建哈希表（使用h₂），扫描相应的R-Partition，并在表中添加元组，同时丢弃副本。

- 更加通用的Join条件

- 几个属性的等式（例如：`R.sid=S.sid AND R.rname = S.sname`）：
 - 对于索引嵌套循环，在`<sid, sname>`上构建索引（如果S是内部的）；或者在`sid`或`sname`上使用现有的索引。
 - 对于排序归并和哈希连接，对两个连接列的组合进行排序/分区。
- 不等式条件（例如：`R.rname < S.sname`）：
 - 对于索引嵌套循环，需要（聚簇）B+树索引。匹配程度可能比等值连接高得多
 - 哈希连接，排序合并连接不适用
 - 基于块的嵌套循环很可能是这里最好的连接方法。

- 聚合操作(AVG、MIN等)

- 无分组情况：一般来说，需要扫描关系；给定一个树索引，其搜索键包含SELECT和WHERE子句中的所有属性，可以执行仅索引扫描。

查询优化 Query Optimization

B 数据库的物理设计

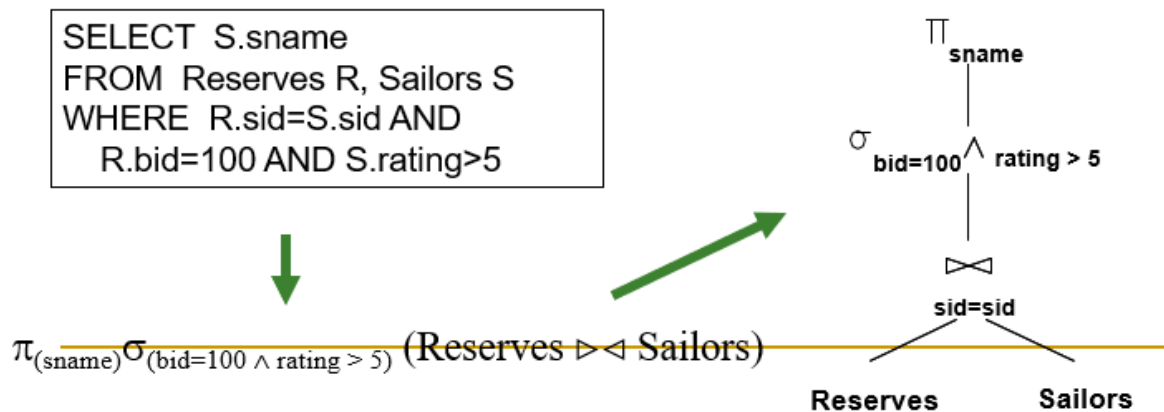
Rule of thumb 经验规则

- 查询优化器可以使用索引、聚簇等。
- 索引的选择：需要考虑涉及哪些表、应使用那些属性作为搜索的键，是否使用多重索引、聚簇
 - 一些方法：依次考虑最重要的查询；考虑使用当前索引所能得到的最优的计划；若能得到更好计划，考虑是否添加额外的索引
 - 索引的影响：索引提高查询速度，但更新表速度变慢；索引需要磁盘空间
 - 需要考虑的问题：
 - WHERE子句中提到的属性是索引搜索键的候选属性时：
 - 范围条件对聚簇很敏感
 - 但精确匹配条件不一定需要聚簇
 - 选择对许多查询有利的索引，但每个关系只能聚集一个索引，所以明智地选择吧！

A 查询优化

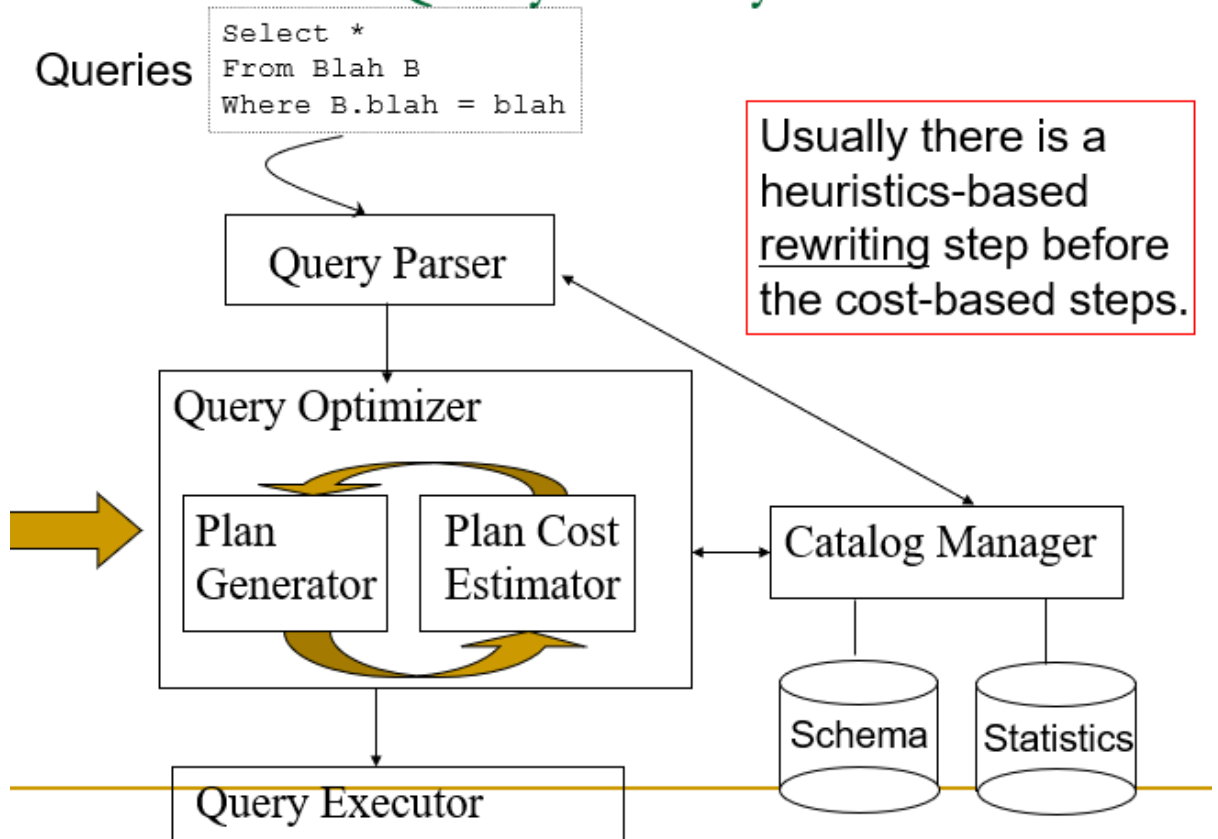
- 查询计划：树的关系代数操作(和其他一些)与选择算法的每个操作。我们找到最理想的计划，预估代价，避免最差的计划。
 - 三个主要问题：对于给定的查询，需要考虑哪些计划？如何估算一项计划的成本？如何在计划空间中搜索好的计划。

- 查询可以转换为关系代数，将关系代数转换为树，连接形成分支，每个操作符都可以指定实现方式，也可以按不同的顺序组织。



- 变换树的各个节点的位置，不同树反映着不同的查询计划，每一个节点还需要具体的指明物理查询方法。
- 一个基于代价估算的查询子系统：

Cost-based Query Sub-System



- 查询优化需要：
 - 封闭的算子集合：关系操作(表入，表出)；封装(例如基于迭代器)
 - 规划空间：基于关系等价，不同的实现
 - 成本估算：基于成本计算公式，或者大小估计，依次根据基表上的目录信息和选择度 Selectivity (减少因子 Reduction Factor)估计
 - 搜索算法：筛选计划空间，找到成本最低的选项

- 节点可指明实现方式
 - On the fly**：一种节点实现方式，不使用中间文件（写到硬盘上）直接推送，好处是减少IO次数
- 选行条件下推 **Push Select**（一般作为缺省的优化步骤）
 - 提前把选行条件下推

Sailors (*sid*: integer, *sname*: string, *rating*: integer, *age*: real)

Reserves (*sid*: integer, *bid*: integer, *day*: dates, *rname*: string)

Reserves: Each tuple is 40 bytes long, 100 tuples per page, 1000 pages.

Assume there are 100 boats

Sailors: Each tuple is 50 bytes long, 80 tuples per page, 500 pages.

Assume there are 10 different ratings

Assume we have 5 pages in our buffer pool!

图1

图2

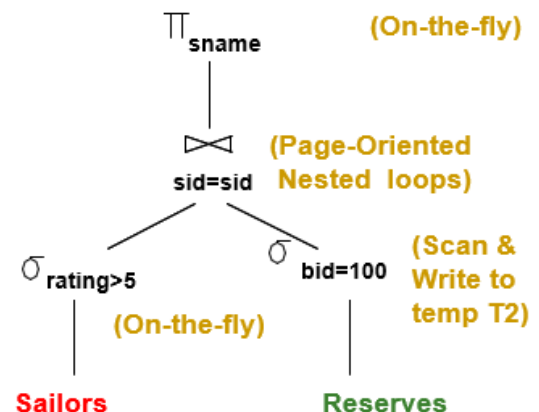
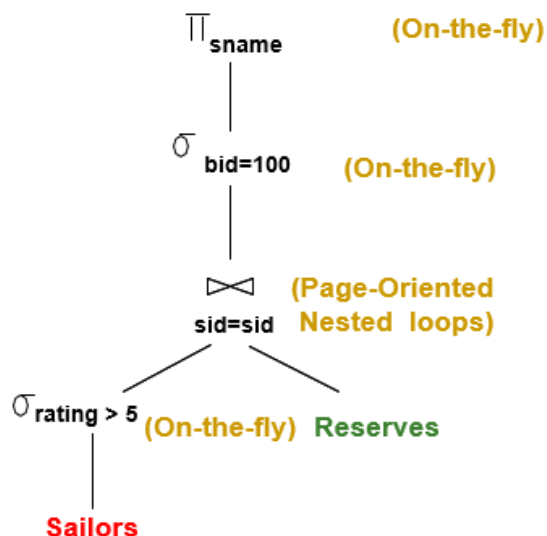
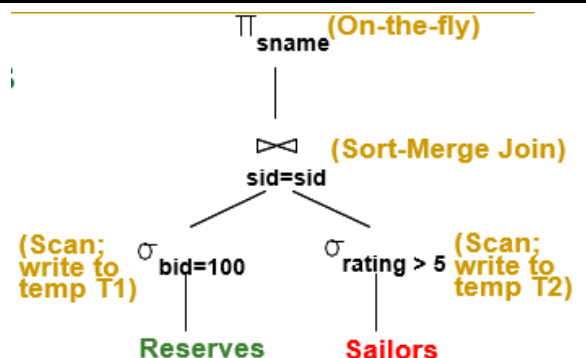
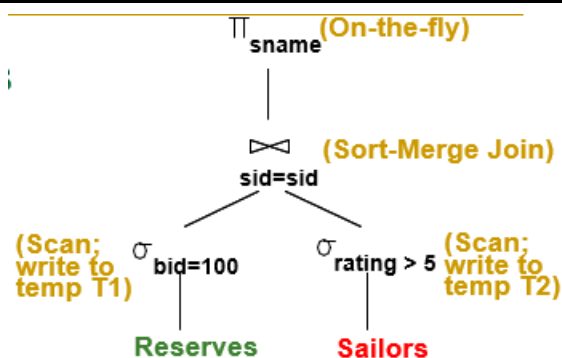


图3

图4



- 如果连接使用的是基于页面的嵌套循环（规定左边为外层）
 - 一个表下推条件（图1）： $500 + 1000 * 500 * 50\% = 250500$

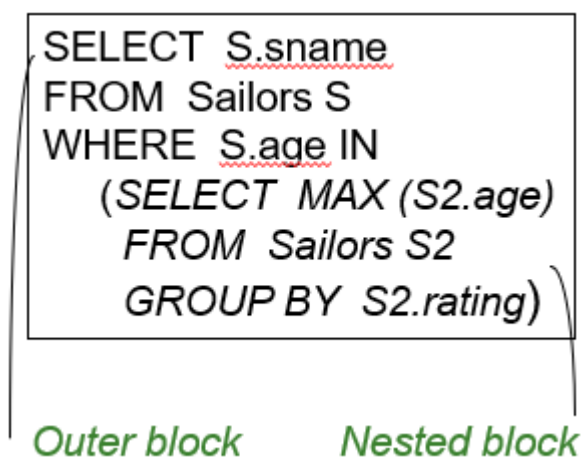
- 两个表下推条件（图2）： $(50\% * 500) * (1\% * 1000) + 500 + 1000 + (1\% * 1000) = 4010$
 - 最后一项是写10个预约表写到硬盘，他的意思是先用所有缓冲区把10个预约表筛选出来出来在放到硬盘里，这样你每次遍历才能保证内层循环里可以直接读10页面。
 - 如果你不去这样做退化到 $(50\% * 500) * (1000) + 500 + 1000$ 了
- 如果使用排序归并连接（图3）
 - 扫描 + 选行并写回 + 外排序 + 归并
 - $(1000 + 500) + (10 + 250) + (2 * 2 * 10 + 2 * 4 * 250) + (10 + 250) = 4060$
- 如果使用基于块的嵌套循环连接（图4，和图三差不多，Sort-Merge改成BNL）
 - 初始扫描并选行 + 写选行结果入磁盘 + 块处理
 - $(1000 + 500) + (250 + 10) + (10 + \text{ceil}(10 / 3) * 250) = 2770$

• 系统 R 和 系统 R风格的查询优化器

- R系统的影响：广泛使用，在连接高达10-15个时效果良好
- Cost估计：
 - 非常不精确，但在实践中是可以的。
 - 用于估计操作成本和结果大小的系统目录中的统计数据。
 - 考虑CPU和I/O成本的组合。
- Plan Space 规划空间：太大，必须修剪。
 - 忽视许多计划中共同的、Cost过高的子树
 - 在某些实现中，只考虑左深计划的空间。
 - 在某些实现中避免了笛卡尔积（Cartesian Products）。

• 查询块 Query Block

- 将查询划分为块，对块内部单独进行优化，不相关的嵌套块都只计算一次，相关的嵌套块调用若干次。
- 相关指内层嵌套块的某些输入需要外层嵌套块来得到且每次不一定相同



- 左深连接树 Left-Deep Join Tree：（逻辑上的一个查询计划）右侧分支始终为基表，每次左下到右上依次连接，需要考虑连接的顺序和连接使用的方法。

- **例子：**对于每一个至少有两个红色船预订的水手，查找水手ID和水手预订红色船的最早日期。

```
SELECT S.sid, MIN (R.day)
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = "red"
GROUP BY S.sid
HAVING COUNT (*) >= 2
```

将SQL转化成关系代数

$$\pi_{S.sid, MIN(R.day)} \left(\begin{array}{l} \text{(HAVING COUNT(*) > 2 (} \\ \text{GROUP BY } \underline{S.Sid} (} \\ \text{\underline{B.color} = "red" (} \\ \text{Sailors } \bowtie \text{Reserves } \bowtie \text{Boats))))) \end{array} \right)$$

- 关系代数的等价性：
 - 选行（尽量挑**选择度**小的放在内层使用）：
 - **级联 Cascade**：多个条件一起选一次行，或多次选行每次只用一个条件（在右边的方案下级联遍历规模会因一次次搜索而减小）
 - **交换 Commute**：先选条件A再选条件B和先选条件B再选条件A是等价变化
 - 选列：
 - **级联 Cascade**： $\pi_{a_1}(R) = \pi_{a_1}(\dots(\pi_{a_1, \dots, a_n}(R)) \dots)$
 - 笛卡尔积 / 自然连接：（不考虑笛卡尔积结果列的顺序）
 - 结合律 Associative 和交换律 Commutative
 - 更多等价条件：
 - **选列与选行交换**：选列条件 可以与 只使用该选列结果的选择条件 进行交换。
 - **笛卡尔积转自然连接**：在笛卡尔积的两个参数对应的属性之间进行的选行 可以转化为 一个自然连接
 - **换行与自然连接的结合**（提前选行，选行下推）：对两张表的自然连接进行选行 等价于 对一张表进行选行再与另一张表进行自然连接（前提是选择条件与另一张表无关）
- 估算代价 Cost Estimation
 - 除了考虑 IO次数，还应考虑 CPU 计算代价，并且给出二者的换算因子
- 一张表的信息：元组数、页面数、最小最大值、一个列的不同取值总数、索引的高度、索引所占的页面数

Statistic	Meaning
NTuples	# of tuples in a table (cardinality)
NPages	# of disk pages in a table
Low/High	min/max value in a column
Nkeys	# of distinct values in a column
IHeight	the height of an index
INPages	# of disk pages in an index

- 规模估算和选择度：（启发式）

- 最大输出行数 Max output cardinality：输入行数的积
- 选择度 Selectivity(Reduction Factor)：输出 / 输入行数
- 实际行数 = 最大输出行数 * 所有表选择度的积
- 选择度估算：
 - $col = val$ ：选择度 = $1 / \text{一个列的不同取值总数}$ （均匀分布的假设）
 - $col_1 = col_2$ ：选择度 = $1 / \max(\text{两表中一个列的不同取值总数})$
 - $col > val$ ：选择度 = $(\text{最大值} - val) / (\text{最大值} - \text{最小值})$

- Term $col_1 = col_2$

- $RF = 1 / \max(NKeys(I_1), NKeys(I_2))$

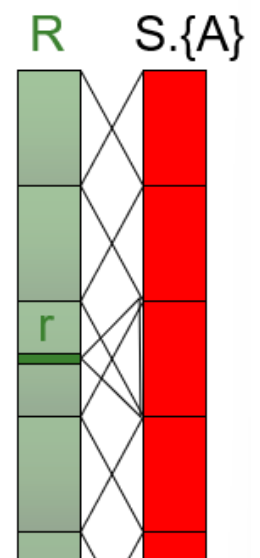
- Q: Given $R \text{ join } S$, what is result cardinality?
Two cases:

1. Key for R, Foreign Key for S

- A common case, treat it specially! $RF = 1/|R|$

2. join on non-key {A}

- For each $r \in R$, $NTuples(S)/NKeys(A,S)$ result tuples
so... $(NTuples(R) * NTuples(S)) / NKeys(A,S)$
 - For each $s \in S$, $NTuples(R)/NKeys(A,R)$
so... $(NTuples(S) * NTuples(R)) / NKeys(A,R)$



- 动态规划生成连接计划：首先要求子计划是最优的
- 单关系计划 Single-relation plans 的代价估计：（还要考虑去重的费用）
 - 索引扫描：树高 + 1
 - 聚簇索引：（索引页数 + 数据页数）* 选择度
 - 非聚簇索引：（索引页数 + 元组总数）* 选择度
 - 顺序扫描：索引页数

例子：SELECT S.sid FROM Sailors S WHERE S.rating=8

- 拼表 Multiple Relations 的优化：

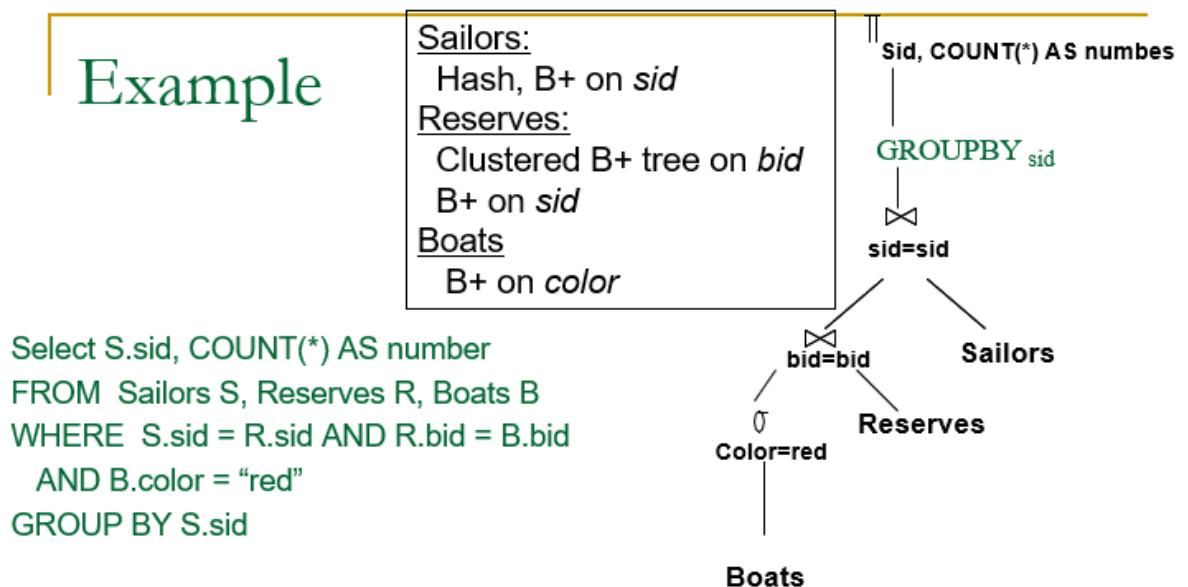
- 只考虑左深树，左深树允许我们生成所有流水线（Fully Pipelined）的计划，中间结果不写入临时文件（但例如排序归并连接必须写中间文件）
- 遗传算法：n 张表对应旅行商问题的 n 个城市（一个序列就可以是一个编码），然后通过遗传算法求得最优解

- 动态规划 Dynamic Programming 过程：不仅只考虑单张表查询的最有计划，因为一些查询语句可能产生一些对之后查询有价值的东西（Interesting order columns），例如GROUP BY的KEY和JOIN的KEY相同时

- **Interesting Orders:** ORDER BY, GROUP BY 和 JOIN 的属性

- 查询优化（物理计划代价估算）：

- 趟 1：为单独获取每个关系表找到最佳方案
- 趟 2：对于第1步中的每个计划，使用所有连接方法（并匹配内部访问方法）生成连接另一个关系作为内部关系的计划，并为每个连接保留最便宜的计划（考虑关系对、顺序等有价值的东西）
- 趟 3+：使用上一趟的计划作为外部关系表，生成下一个连接的计划，考虑 ORDERBY / GROUPBY / AGGREGATE 增加的成本，最后选择最便宜的方案。
 - 如果存在 Interestingly Ordered Plan 匹配，则增加成本为 0
 - 如果通过一个额外的排序/哈希，那还得计算增加成本
- 例子：



- 趟 1：找到每一个关系表的 Best plan（还没有考虑连接）
 - 对于 Reservers 和 Sailors 是 直接扫描
 - 对于 Boats 是 建立在 Color 上的 B+是树
- 趟 2：对于趟 1中的每个计划，使用所有连接方法(以及匹配内部访问方法)生成连接另一个关系的计划作为内部关系，然后找到每一对连接的最佳计划。

```

File Scan Reserves (outer) with Boats (inner)
File Scan Reserves (outer) with Sailors (inner)
File Scan Sailors (outer) with Boats (inner)
File Scan Sailors (outer) with Reserves (inner)
Boats Btree on color with Sailors (inner)
Boats Btree on color with Reserves (inner)

```

- 趟 3+: 使用上一趟计划作为外部关系，生成下一次连接的计划
 - 例如：两个连接都用排序归并连接（好像会产生有序SID，这样GROUPBY就没有增加成本了）

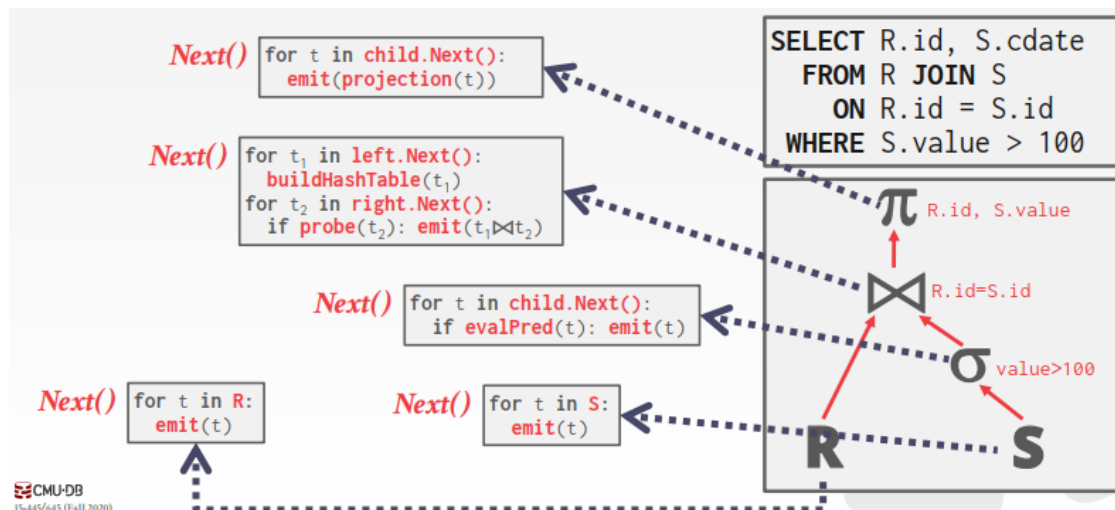
然后，添加 GROUP BY / AGGREGATE 的成本：按Sid对结果排序的代价，除非它已经被之前的操作符排序过，最后，选择最便宜的计划

查询计划的物理实现

- Processing Models: 定义系统如何执行查询计划

- 迭代器模型 Iterator Model

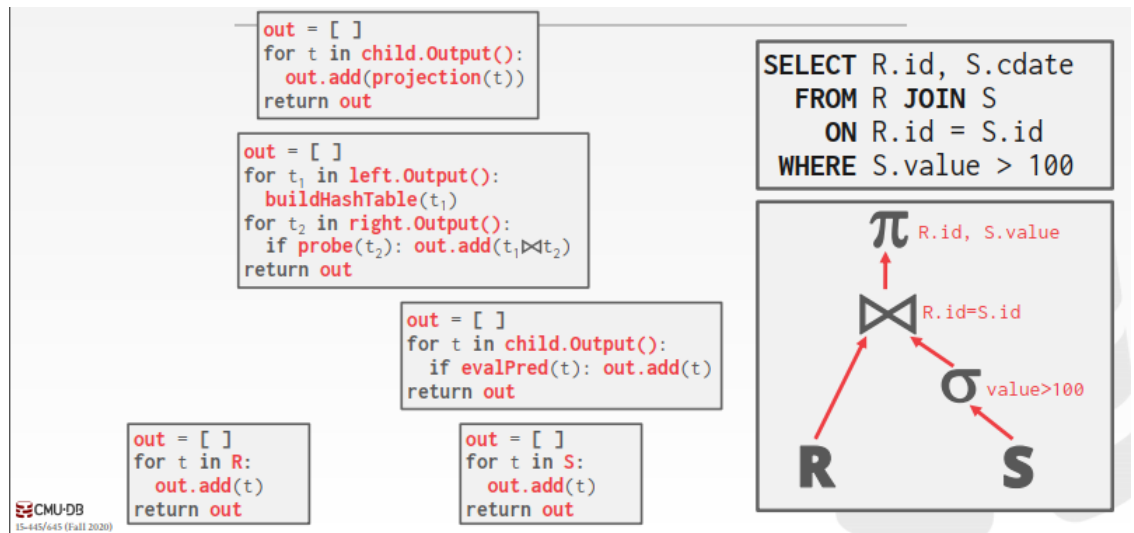
- 实现Next()接口：每个查询计划操作符实现一个Next函数，每次调用时，操作符要么返回单个元组，要么返回空标记(如果没有更多元组)。操作符实现了一个循环，在它的子对象上调用next来检索它们的元组，然后处理它们。



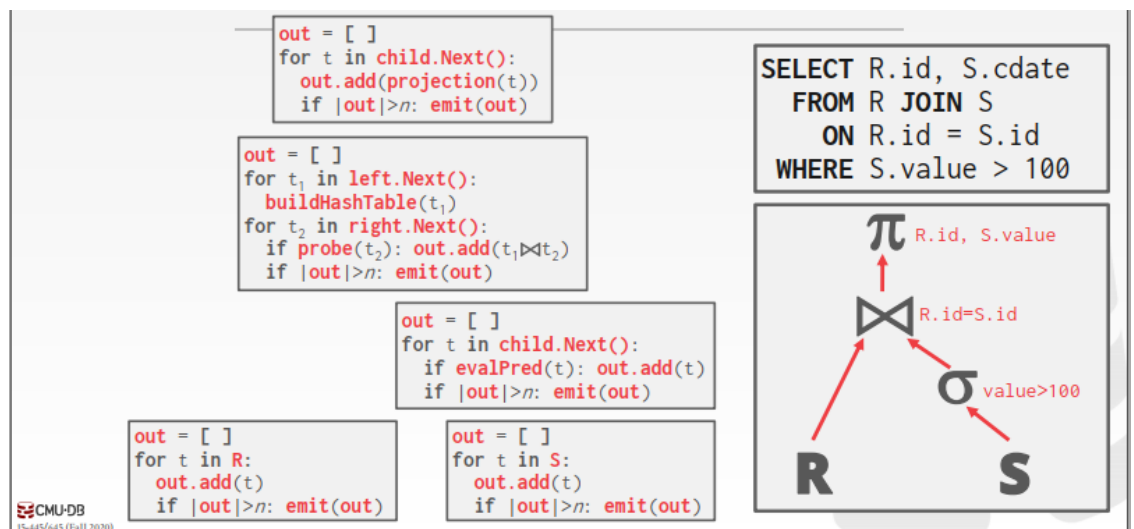
- 一次一条元组，返回一条元组？频繁的函数调用

- 物化模型 Materialization Model、

- 每个操作符一次处理所有的输入，然后一次发出所有的输出。运算符将其输出“具体化”为单个结果，数据库管理系统可以将提示下推，以避免扫描过多的元组，可以发送一个实体化行或单个列。



- 对所有输入一次性处理然后输出所有结果？对空间的需求？、
- 矢量化/批模型 Vectorized / Batch Model
 - 类似于迭代器模型，每个操作符在该模型中实现一个Next函数。每个操作符发出一批元组，而不是单个元组。运算符的内部循环一次处理多个元组，批量的大小可以根据硬件或查询属性的不同而不同。



- 放一部分，自顶向下和自底向上